ALA - 2 Python Deep Dive: Interview Intensive

Phase 1: Conceptual Understanding

Question 1: In your own words, why do we use try and except instead of just writing 'perfect' code?

The Interviewer is looking for : An understanding of "Runtime Errors" versus "Syntax Errors."

- Response: Even perfect logic can fail due to external factors we can't control, like a user entering a string when we expect a number, a missing file, or a dropped network connection. try and except allow the program to handle these 'exceptions' gracefully without crashing the entire application.

Question 2: "What is the danger of using a bare except: clause?"

The Interviewer is looking for: Knowledge of debugging and system-level interrupts.

- **Response:** A bare except: catches *every* error, including KeyboardInterrupt (Ctrl+C) and SystemExit. This makes it nearly impossible to debug because you don't know which specific error occurred. It's always best practice to catch specific exceptions like ValueError or ZeroDivisionError so you can handle them appropriately.

Phase 2: The Logic Flow

Question 3: "Explain the difference between the else and finally blocks."

The Interviewer is looking for: Clarity on the execution path.

- **Response:** * else: This code runs only if no exceptions were raised in the try block. It's great for code that should only execute if the 'try' was successful.
    - **finally:** This code always runs, regardless of whether an exception occurred or was caught. It is primarily used for 'cleanup' tasks like closing a file or a database connection.

Question 4: "If a try block has a return statement, does the finally block still execute?"

The Interviewer is looking for: Advanced knowledge of Python's execution priority.

- **Response:** "Yes. Python guarantees that the finally block will execute before the function actually returns the value to the caller. This is a unique feature to ensure resources are never left open."

Phase 3: Practical Scenario

Coding Task: "Write a function that safely converts a list of strings to integers."

Interviewer Goal: See if you can apply the logic to a collection.

CODE:
```
def safe_convert(data):
    results = []
    for item in data:
        try:
            results.append(int(item))
```

```
        except (ValueError, TypeError):
            print(f"Warning: '{item}' is not a valid number. Skipping.")
        except Exception as e:
            print(f"An unexpected error occurred: {e}")
    return results

my_list = ["10", "20", "Apple", "30"]
print(safe_convert(my_list))
```

Output: Warning: 'Apple' is not a valid number. Skipping. [10, 20, 30]

Step-by-Step Execution Breakdown

1. **Iteration 1 ("10"):** The try block successfully converts the string "10" to the integer 10 and appends it to results.
2. **Iteration 2 ("20"):** Same as above; the integer 20 is added to the list.
3. **Iteration 3 ("Apple"):** * The code int("Apple") fails and triggers a ValueError.
    - Python immediately jumps to the first except block: except (ValueError, TypeError):.
    - It prints the Warning message to the console.

- Because the error was "caught," the loop does not crash; it simply moves to the next item.

4. **Iteration 4 ("30"):** The try block successfully adds 30 to the list.
5. **Final Print:** The function returns [10, 20, 30], which is then printed by the final line of your script.

➢ In an interview, you would point out that without the try-except block, the program would have crashed at "Apple" with a ValueError: invalid literal for int() with base 10: 'Apple', and the number 30 would never have been processed. This code demonstrates robustness.